

FPC-16 Fixed-Point Precision Core

Extending the StarCore-1 RISC Processor with Native Q8.8 Fixed-Point Arithmetic

Group 14 | May Yuan-Klitzner (YNKMAY001) | Colby Skirving (SKRCOL001) | Sandakahle Bhengu (BHNSAN010) | EEE4120F HPES 2026 | University of Cape Town

Introduction The StarCore-1 is a 16-bit single-cycle Harvard RISC processor designed for the OrbitEdge-1 CubeSat mission. FPC-16 is our extension that adds native fixed-point multiply and divide directly into the processor hardware.

Motivation

- StarCore-1 handles Q8.8 addition and subtraction natively
- No native multiply or divide instruction exists in the ISA
- FPC-16 adds single-cycle FPMUL and FPDIV, focusing hardware effort where software cannot keep up

FPC-16

Fixed-Point Precision Core

Native fixed-point arithmetic for the OrbitEdge-1 mission computer

Group 14 | EEE4120F HPES 2026 | University of Cape Town

Faster. Smarter. Mission-Ready.

MOTIVATION

The StarCore-1 handles Q8.8 addition and subtraction natively, but has no multiply or divide instruction. FPC-16 addresses this by adding single-cycle native FPMUL and FPDIV hardware, focusing effort exclusively where software cannot keep up.

KEY FEATURES

- ✓ **FPMUL (opcode 1110)**: 32-bit intermediate product, bits[23:0] extracted
- ✓ **FPDIV (opcode 1111)**: Pre-shifted dividend, divide-by-zero protected
- ✓ **Signed Arithmetic**: Two's complement FPMUL & FPDIV support
- ✓ **Memory-Mapped I/O**: 0x00FE input / 0x00FF output via LD/ST
- ✓ **End-to-End Pipeline**: Sensor read → FPMUL/FPDIV → telemetry output
- ✓ **Backward Compatible**: All original StarCore-1 tests pass

PERFORMANCE (Amdahl's Law Analysis)

46-87x
instruction reduction
1 FPMUL vs software
shifts-and-add loop

up to 2x
system speedup
at 50% FPC workload
(Amdahl's Law)

VERIFICATION RESULTS — 123 / 123 Tests Passing		
Testbench	Tests	Covers
ALU_tb	42 / 42	FPMUL, FPDIV, Signed arithmetic
ALU_Control_tb	18 / 18	New opcode decoding
ControlUnit_tb	16 / 16	FPMUL/FPDIV control signals
GPR_tb + InstMem_tb + DM_tb	42 / 42	Register file & memory modules
StarCore1_tb	5 / 5	Full I/O pipeline verified

TARGET APPLICATION

OrbitEdge-1 CubeSat — 500 km low Earth orbit
Altitude control | Telemetry checksums | Atmospheric sensing
Power budget: < 400 mW | Radiation-tolerant FPGA fabric

May Yuan-Klitzner | Colby Skirving | Sandakahle Bhengu — Group 14, EEE4120F 2026

Entire System Overview



ALU, ALU_Control, ControlUnit, Datapath,
StarCore1, DataMemory

GPR, InstructionMemory (logic)

FPMUL → 1110, FPDIV → 1111
ALU Cnt extended 3-bit → 4-bit

0x00FE → input port
0x00FF → output port

All original StarCore-1 tests still pass

Q8.8 Fixed-Point Format and ISA Extension

Custom Processor Explanation - Part 1 of 2

Q8.8 Fixed-Point Format

Bit Field	Bits	Range / Precision
Integer part	[15:8]	0 to 255
Fractional part	[7:0]	0 to 255/256
Precision	1/256	≈0.004
Max value	0xFFFF	255.996

Key Design Decision: FPADD and FPSUB reuse existing opcodes 0010 and 0011 as binary addition and subtraction are identical for Q8.8. Hardware budget is spent only on FPMUL and FPDIV.

ISA Extension

Operation	Opcode	ALU_Cnt	New HW?
FPADD	0010 (reuse)	0000	No
FPSUB	0011 (reuse)	0001	No
FPMUL	1110 (new)	1000	Yes
FPDIV	1111 (new)	1001	Yes

ALU_Cnt Extended: All 8 original 3-bit ALU control codes were occupied. ALU_Cnt was extended from 3-bit to 4-bit, assigning 4'b1000 = FPMUL and 4'b1001 = FPDIV.

FPMUL, FPDIV and Signed Arithmetic

Custom Processor Explanation - Part 2 of 2

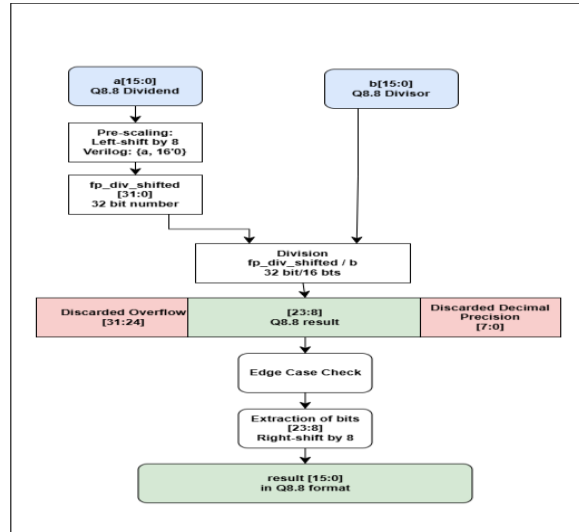
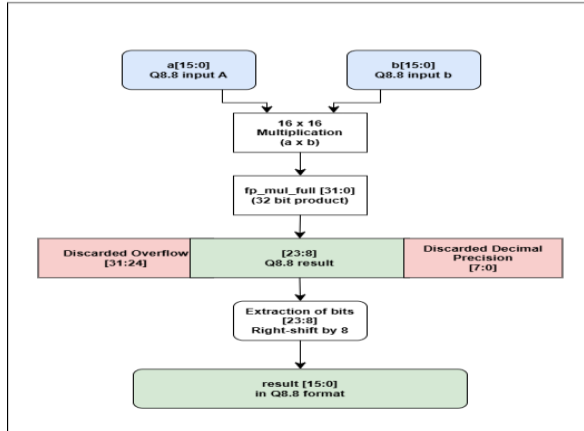


Fig. 2. Q8.8 fixed-point arithmetic datapath for FPMUL and FPDIV. Left: FPMUL. Right: FPDIV

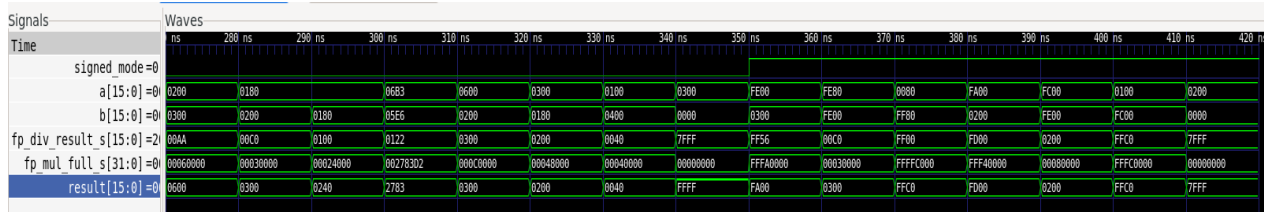


Fig. 3 GTKWave simulation waveform of ALU_tb (270-420ns) showing unsigned Q8.8 FPMUL and FPDIV (signed_mode=0, left) transitioning to signed two's complement operations (signed_mode=1, right) at 350ns

FPMUL - Q8.8 Multiply

- $a \times b \rightarrow 32\text{-bit intermediate product}$
- Extract bits[23:8] $\rightarrow$ correct Q8.8 result
- bits[31:24] = overflow (discarded)
- bits[7:0] = sub-precision (discarded)

FPDIV - Q8.8 Divide

- Pre-shift dividend left by 8: $\{a, 8'b0\}$
- Divide: $\{a, 8'b0\} / b$
- Result is directly the correct Q8.8 quotient

Signed Arithmetic

- signed_mode flag selects two's complement path in the ALU
- Enables negative sensor readings and attitude angles
- Signed $\div 0$ saturates to 0x7FFF (max positive signed)
- Unsigned $\div 0$ saturates to 0xFFFF (max unsigned)

Gold Standard Output - Unsigned

Expected Q8.8 results calculated manually and verified in Python

Unsigned FPMUL

Operation	Expected	Hex	Pass
$2.0 \times 3.0 = 6.0$	6.0	0x0600	✓
$1.5 \times 2.0 = 3.0$	3.0	0x0300	✓
$1.5 \times 1.5 = 2.25$	2.25	0x0240	✓
$6.7 \times 5.9 = 39.51^*$	39.51	0x2783	✓

* 39.53 exact - Q8.8 truncation error expected

Unsigned FPDIV

Operation	Expected	Hex	Pass
$6.0 \div 2.0$	3.0	0x0300	✓
$3.0 \div 1.5$	2.0	0x0200	✓
$1.0 \div 4.0$	0.25	0x0040	✓
any $\div 0$	saturate	0xFFFF	✓

“This is our gold standard — expected Q8.8 results calculated independently of hardware implementation. Simulation must match these exactly.”

Gold Standard Output - Signed

Two's complement Q8.8 fixed-point arithmetic

Signed FPMUL

Operation	Expected	Hex	Pass
-2.0×3.0	-6.0	0xFA00	✓
-1.5×-2.0	3.0	0x0300	✓
0.5×-0.5	-0.25	0xFFC0	✓

Signed FPDIV

Operation	Expected	Hex	Pass
$-6.0 \div 2.0$	-3.0	0xFD00	✓
$-4.0 \div -2.0$	2.0	0x0200	✓
$1.0 \div -4.0$	-0.25	0xFFC0	✓
any $\div 0$	+MAX sat	0x7FFF	✓

Note on Truncation Error: $6.7 \times 5.9 = 39.53$ exactly, but hardware produces 39.51. This is expected and inherent to Q8.8 fixed-point representation as input values 6.7 and 5.9 cannot be represented exactly in Q8.8 format. This is not a hardware bug.

Divide-by-Zero Protection: Unsigned $\div 0 \rightarrow 0xFFFF$ (maximum unsigned value) | Signed $\div 0 \rightarrow 0x7FFF$ (maximum positive signed value). Both verified in testbench.

Gold Standard Explanation

How the gold standard was independently derived

1

Manual Q8.8 Calculation

Convert decimal to Q8.8: multiply by 256 and truncate.

Example: $6.7 \times 256 = 1715.2 \rightarrow$ truncated to 1715 = 0x06B3

Multiply integers: $1715 \times 1510 = 2,589,650$

Extract bits[23:8]: $2,589,650 \div 256 = 10,115 = 0x2783 = 39.51$

2

Python Verification

A Python script independently implementing Q8.8 arithmetic from first principles confirmed all 15 expected values.

Cross-checked against manual calculation before any hardware implementation.

Ensures gold standard is not influenced by hardware design choices.

3

Known Truncation Error

$6.7 \times 5.9 = 39.53$ exactly but Q8.8 encoding of 6.7 $\rightarrow$ 0x06B3 (6.6992) and 5.9 $\rightarrow$ 0x05E6 (5.8984) introduces input truncation.

Hardware correctly produces $39.51 = 0x2783$. Confirmed by Python script.

Key Point: Gold standard established independently of hardware implementation - ensures genuine verification baseline.

Simulation Output and Verification

123 / 123 tests passing across all testbenches

```
--- FPMUL (alu_control = 4'b1000) ---  
PASS [T28]: result=1536 (0x0600) ← 2.0×3.0=6.0  
PASS [T31]: result=10115 (0x2783) ← 6.7×5.9=39.51  
  
--- FPDIV (alu_control = 4'b1001) ---  
PASS [T32]: result=768 (0x0300) ← 6.0÷2.0=3.0  
PASS [T35]: result=65535 (0xffff) ← any÷0 saturates  
  
--- FPMUL Signed (signed_mode=1) ---  
PASS [T36]: result=64000 (0xfa00) ← -2.0×3.0=-6.0  
PASS [T37]: result=768 (0x0300) ← -1.5×-2.0=3.0  
  
--- FPDIV Signed (signed_mode=1) ---  
PASS [T39]: result=64768 (0xfd00) ← -6.0÷2.0=-3.0  
PASS [T42]: result=32767 (0x7fff) ← any÷0 saturates  
  
--- I/O Port Verification ---  
io_out = 0x0280  
PASS [T1]: value=0x0280 ← sensor value captured!  
=== ALL 123 TESTS PASSED ===
```

Listing 1: Selected ALU_tb and StarCore1_tb simulation output confirming correct Q8.8 FPMUL, FPDIV, signed arithmetic, divide-by-zero saturation, and I/O pipeline — 123/123 tests passing

Test Summary

Testbench	Result
ALU_tb	✓ 42/42
ALU_Control_tb	✓ 18/18
ControlUnit_tb	✓ 16/16
GPR+InstrMem+DM	✓ 42/42
StarCore1_tb	✓ 5/5
TOTAL	✓ 123/123

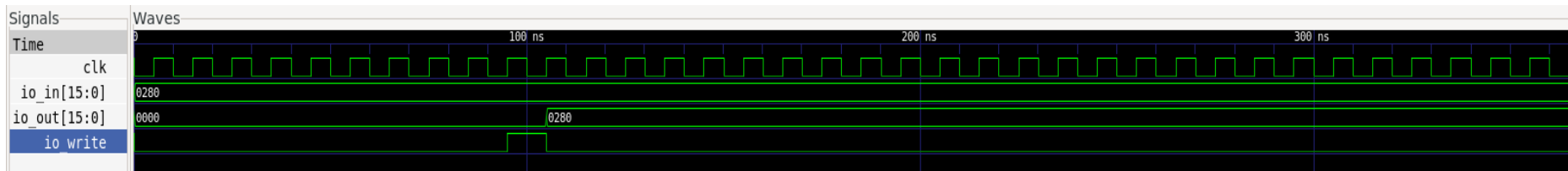
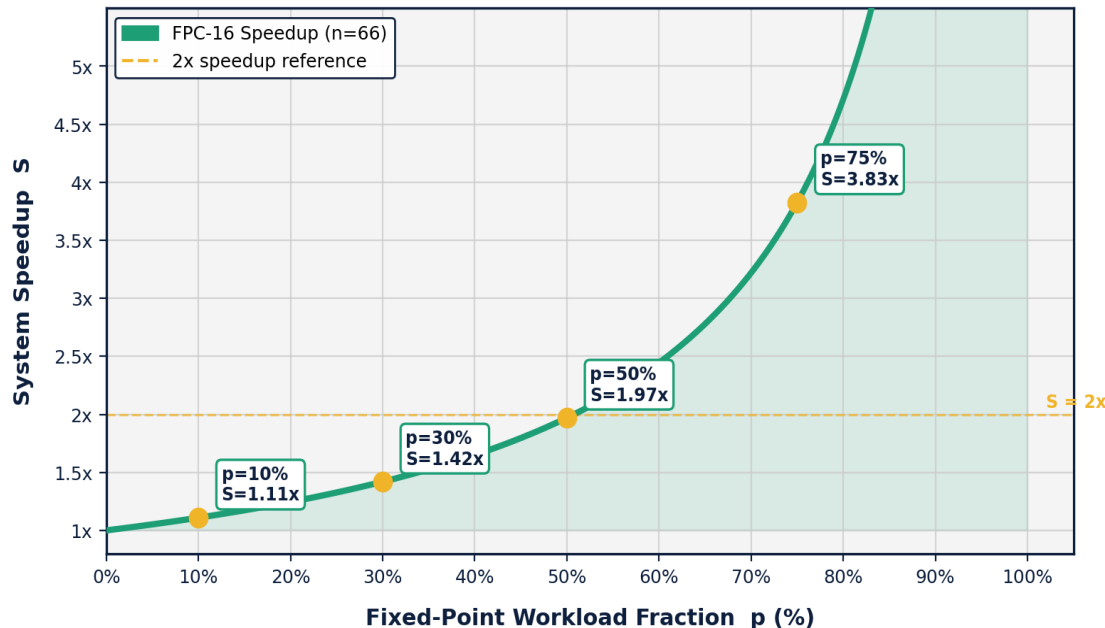


Fig.4: GTKWave waveform of StarCore1_tb showing io_in=0x0280 captured to io_out on io_write pulse, confirming correct memory-mapped I/O pipeline.

Performance Analysis

Amdahl's Law - FPC-16 vs Software Baseline

Amdahl's Law — FPC-16 System Speedup vs FP Workload
n = 66 (midpoint of 46-87x instruction reduction)



$$S = 1 / (1 - p + p/n)$$

p = FP workload fraction, n = 66
(midpoint of 46 - 87x)

FP Workload (p)	Speedup (S)
10%	1.11x
30%	1.42x
50%	1.97x ≈ 2x
75%	3.83x

- StarCore-1 has no native multiply - software FPMUL requires shift-and-add loops (~46 - 87 instructions)
- Hardware FPMUL: 1 single-cycle instruction
- Speedup scales directly with FP workload fraction

Summary and Future Work

What FPC-16 Delivers

- ✓ Native Q8.8 FPMUL and FPDIV in single-cycle hardware
- ✓ Signed two's complement arithmetic support
- ✓ Memory-mapped I/O: sensor read → compute → telemetry output
- ✓ 123 / 123 tests passing across all testbenches
- ✓ Fully backward compatible - all original StarCore-1 tests pass

Future Work

- Expose signed_mode through processor ISA
- Extend to Q10.6 format (integer range 0-1023)

GitHub Repository: [\[EEE4120F-Project Repository\]](#) - Contains: Verilog source, testbenches, test vectors, simulation outputs and waveforms

Thank you - we welcome any questions